

TeamTask – Documentación Técnica

Versión 1.0 | Flutter + Supabase | Android

1. Descripción del Proyecto (TT-139)

TeamTask es una aplicación móvil de gestión de tareas en equipo, desarrollada con Flutter. Permite a los usuarios crear tableros de trabajo, asignar tareas a miembros del equipo, recibir notificaciones en tiempo real y visualizar estadísticas de progreso. El backend está implementado sobre Supabase, que proporciona autenticación, base de datos PostgreSQL y comunicación en tiempo real.

1.1 Tecnologías utilizadas

Tecnología	Versión	Propósito
Flutter	3.x	Framework principal para desarrollo móvil multiplataforma
Dart	3.x	Lenguaje de programación
Supabase	2.x	Backend: autenticación, base de datos y tiempo real
Riverpod	2.x	Gestión de estado de la aplicación
go_router	latest	Navegación declarativa entre pantallas
fl_chart	latest	Gráficos de estadísticas
flutter_local_notifications	latest	Notificaciones locales en el dispositivo
font_awesome_flutter	latest	Iconografía adicional

1.2 Requisitos para ejecutar el proyecto

Requisito	Detalle
Flutter SDK	3.0 o superior — https://flutter.dev/docs/get-started/install
Dart SDK	Incluido con Flutter
Android Studio	Para compilar en Android
Cuenta Supabase	Crear proyecto en https://supabase.com
Variables de entorno	Ver sección 1.3

1.3 Variables de entorno (Supabase)

Editar el archivo lib/constants/supabase_constants.dart con las credenciales del proyecto:

```
class SupabaseConstants {
  static const String supabaseUrl = 'https://TU_PROYECTO.supabase.co';
  static const String supabaseAnonKey = 'TU_ANON_KEY';
}
```

1.4 Cómo instalar y correr el proyecto

1. Clonar el repositorio: git clone <URL_DEL_REPOSITORIO>

2. Entrar a la carpeta: `cd teamtask`
3. Instalar dependencias: `flutter pub get`
4. Configurar Supabase en `lib/constants/supabase_constants.dart`
5. Correr la app: `flutter run`
6. Para build de producción Android: `flutter build apk --target-platform android-arm64 --release`
7. Para build de producción iOS: `flutter build ipa --release`

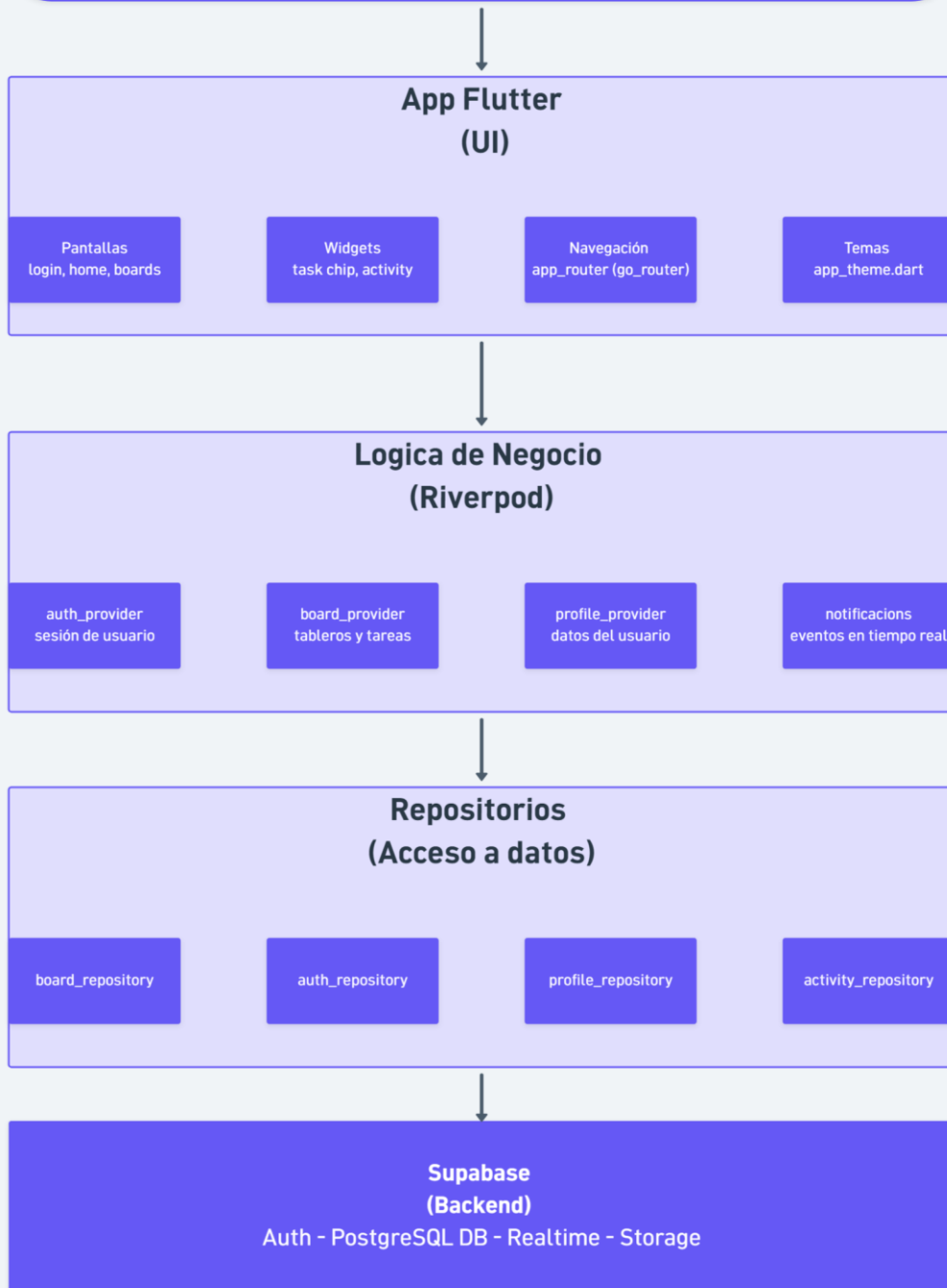
2. Arquitectura del Sistema

La aplicación sigue una arquitectura en capas (Layered Architecture) con separación clara entre presentación, lógica de negocio y acceso a datos.

2.1 Capas de la arquitectura

Capa	Archivos principales	Responsabilidad
Capa de Presentación (UI)	screens/, widgets/	Pantallas y widgets de Flutter. Se comunican únicamente con los Providers de Riverpod.
Capa de Lógica de Negocio	auth_provider.dart, board_provider.dart, profile_provider.dart, notifications/	Providers de Riverpod que manejan el estado global y la lógica de la aplicación.
Capa de Repositorios	board_repository.dart, auth_repository.dart, profile_repository.dart, activity_repository.dart	Clases que abstraen el acceso a Supabase. Los providers consumen estas clases.
Backend (Supabase)	Supabase Cloud	Autenticación (GoTrue), base de datos PostgreSQL, Realtime para notificaciones en vivo.

Diagrama de arquitectura TeamTask



2.2 Flujo de datos

UI (Widget) → Provider (Riverpod) → Repository → Supabase API → PostgreSQL / Auth / Realtime

El flujo inverso (respuesta del servidor) viaja desde Supabase → Repository → Provider (actualiza estado) → UI (se redibuja automáticamente).

2.3 Estructura de carpetas

```
lib/
├── constants/           # Configuración Supabase
├── screens/             # Pantallas de la app
├── widgets/            # Widgets reutilizables
├── notifications/      # Servicio y listeners de notificaciones
├── *_provider.dart      # Estado global (Riverpod)
├── *_repository.dart    # Acceso a datos (Supabase)
├── app_router.dart      # Configuración de navegación
├── app_theme.dart       # Tema visual de la app
└── main.dart           # Punto de entrada
```